SECURERELEASE / PRODUCT DEMO

Build your app. Know what to fix before you ship.

Security readiness for individual developers and small startups.
One workflow for your website, API and source code.

```
securerelease init
securerelease run
```

THE PROBLEM / WHO FEELS IT

Small teams ship software. Security expertise does not ship with it.

Solo developers and early-stage startups need an accessible first security check before deployment.

Knowledge gap

Which checks matter? How do we test authentication, dependencies and browser behavior?

Fragmented workflow

Free tools exist, but installing several engines and interpreting different outputs takes effort.

Real cost

Time away from building, paid-tool or specialist budgets, and rework when issues surface late.

Our opportunity: reduce setup and interpretation friction, not claim that existing security tools are all paid.

Customer hypothesis: teams will value one guided release-check workflow. Time savings, willingness to pay and adoption still need validation.

ONE MAIN FLOW / FROM RUNNING APP TO ACTION

Configure once. Scan. Review. Improve.

1. **Start your app**
   Frontend and backend run in separate terminals.

2. **Run securerelease init**
   Enter UI/API URLs, OpenAPI spec, source folder and test credentials.

3. **Run securerelease run**
   Execute the configured checks and write one HTML report.

4. **Review evidence, fix, rerun**
   Start with coverage gaps and high-impact findings.

What the live demo shows

Scan the original app, then repeat the same workflow against the improved app.

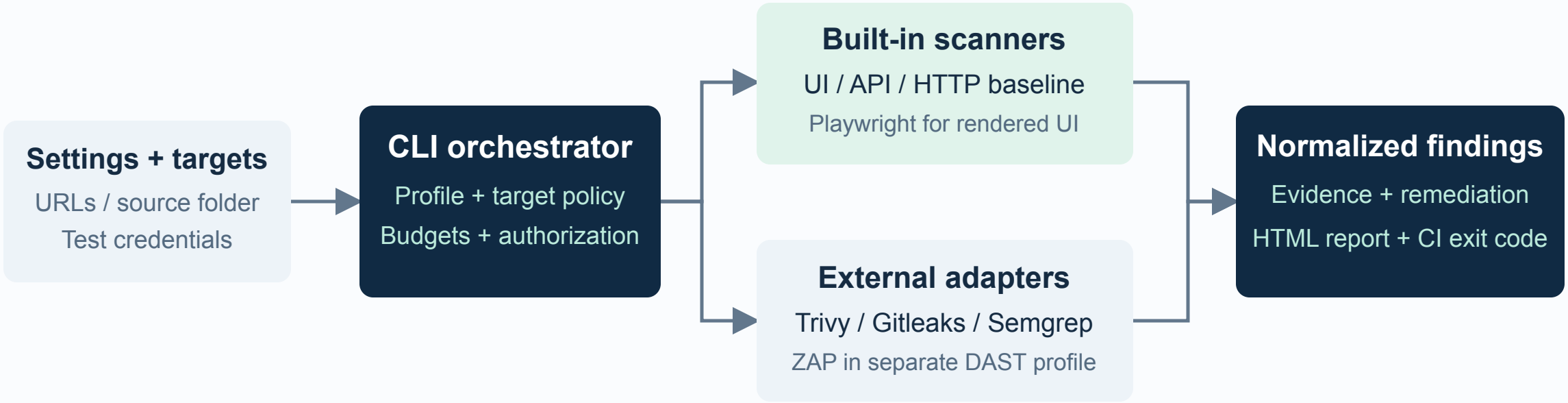
- Compare evidence, not just finding counts.
- Show authentication, headers, ownership and rate-limit signals.
- Keep inherited dependency issues visible.

Same default report path: reports/securerelease-report.html. The next run overwrites it.

Prerequisites: installed engines, running targets and valid test credentials

SIMPLE ARCHITECTURE / ONE ORCHESTRATOR, MULTIPLE CHECKS

Unify proven tools. Add app-level context.



Local-first, not offline-only: the CLI runs locally; external engines may fetch rules or vulnerability databases and follow their own network behavior.

Useful coverage without unsafe defaults.

Build vs. integrate

Compared: write every security engine or wrap established tools.

Choice: integrate Trivy, Gitleaks and Semgrep; build UI/API checks.

Tradeoff: faster breadth, but engine installation and output normalization remain necessary.

Crawler vs. browser

Problem: a plain HTTP crawler cannot see React-rendered login forms.

Choice: Playwright for JavaScript UIs; keep the lightweight crawler.

Tradeoff: better rendered-page coverage, but browser setup and runtime cost.

Coverage vs. safety

Compared: probe every method or require explicit authorization.

Choice: gate active tests and writes; never send DELETE.

Tradeoff: some checks remain skipped. Report that honestly rather than mark them PASS.

Reliability improvement: browser login now waits for the password form to disappear and checks that the session survives a target reload.

NEXT / PRODUCT ROADMAP AND FIRST USERS

Make the first security check part of every small team's release.

Planned product work

- Simpler installation, environment diagnostics and token refresh workflows.
- Better source exclusions, rule relevance and finding prioritization.
- Request-body fuzzing and deeper authenticated workflows.
- Saved baselines, report comparisons and easier CI onboarding.

Planned user acquisition

- Publish a reproducible demo and beginner-friendly setup guide.
- Recruit solo builders and small startup design partners.
- Share through GitHub and developer communities; learn from onboarding failures.
- Measure time to first useful report, repeat scans and verified fixes.

The ask: try it on an app you own and tell us what blocks your next release.